Assignment 3: Benchmarking Approximate Nearest Neighbour Search

CS4.406: Information Retrieval & Extraction

Individual · Due: 25 October 2026, 23:59 IST · Submit on Moodle

Overview

Finding the stored vectors closest to a query is the core step of dense retrieval. Doing it exactly means comparing the query against every vector, which gets too slow at scale. Approximate nearest neighbour (ANN) indexes are much faster, at the price of sometimes missing a true neighbour.

In this assignment you will measure that trade-off yourself. You will benchmark three popular index types, HNSW, IVF-Flat and IVF-PQ, on two public datasets, run one experiment of your choice from a short list, carry out one open-ended experiment of your own on data you generate, and write a short report explaining what you found. We care about careful measurement and clear explanations, not about which index turns out fastest.

Data

Use SIFT (1M image descriptors, 128 dimensions, Euclidean distance) and GloVe-100 (1.18M word vectors, 100 dimensions, angular distance) from ann-benchmarks: SIFT and GloVe-100. Each file contains the vectors to index, 10,000 queries, and the true nearest neighbours of each query. Normalise the GloVe vectors and use Euclidean distance for everything.

Split the queries in half: tune anything you need on the first 5,000, and report your final numbers on the last 5,000.

Part 1: Exact baseline

Build exact search with FAISS and use it as your reference. Write your own recall@10 function, taking care that ties at the 10th neighbour are not counted as mistakes, and add a few unit tests for it. Check that your exact results agree with the neighbours stored in the data file. Finally, time exact search both one query at a time and with all queries in one batch, and explain in a couple of sentences why the two differ so much.

Part 2: Benchmark three indexes

On both datasets, benchmark HNSW, IVF-Flat and IVF-PQ. For each index, try a few build settings (for example M for HNSW, `nlist` for IVF) and, for each, sweep the search-time parameter (`efSearch` or `nprobe`) so that recall ranges from low up to about 0.95 or higher. Measure speed in queries per second, one query at a time on a single thread, and repeat each timing a few times and take the median.

Then report:

- a plot per dataset of recall@10 against queries per second, showing the best trade-off curve for each index and the exact baseline;
- a table of the queries per second each index achieves at recall 0.90 and 0.95;
- a table of build time, index size and bytes per vector for each index at recall 0.95;

- a sentence or two on why any index fails to reach high recall, if one does.

Part 3: Experiment on the standard data

Pick one of the following. Before you run it, write down a short hypothesis and commit it to git, then say in your report whether the results supported it.

- **Scaling.** On SIFT, repeat the benchmark at 100K, 300K and 1M vectors and see how the cost of reaching recall 0.95 grows for HNSW, IVF-Flat and exact search.
- **Query difficulty.** On GloVe-100, measure how hard each query is (for example by its relative contrast), group the queries into four difficulty bins, and compare the recall each index achieves in each bin.
- **Compression.** On SIFT, vary the number of PQ sub-vectors and compare plain IVF-PQ with IVF-PQ followed by exact re-ranking of the top 100 candidates, in terms of recall, speed and memory.

Part 4: Experiment on your own data

The previous parts use data someone else chose. Here you create data that makes an index struggle, and show how and why. Pick your own question about one trade-off or failure mode, different from your Part 3 experiment, and study it in at least two of the three indexes. Your experiment must use data that you generate yourself, either from scratch or by transforming the real vectors.

Some ideas, none of them required:

- **Drifting data.** Build an index on one distribution, then add vectors or send queries from a shifted one (moved mean, rotation, new clusters appearing). Which index degrades, and does it recover if you retrain or rebuild?
- **Update cost.** After building, insert or delete a growing fraction of the vectors. How do insertion time, recall and speed change, and when does rebuilding from scratch become cheaper?
- **Memory limits.** Fix a RAM budget and find the best recall and speed each index can reach inside it, for example by changing the PQ code size or the HNSW degree.
- **Hard geometry.** Control the structure of the data: the dimension, how tight or unbalanced the clusters are, or how many near-duplicates there are. For instance, with uniform random vectors, nearest neighbours become barely closer than everything else as the dimension grows. What does that do to each index?

Whatever you choose, change one thing at a time. Use at least three settings of the property you are varying, plus an unchanged control, so that any difference can be traced to the property. As in Part 3, write down a short hypothesis (what you expect to break, and why) and commit it to git before you run the experiment. A hypothesis that turns out wrong is fine; we grade the reasoning and the care of the experiment, not whether you guessed right.

Keep it small: use at most 500K synthetic vectors, and make your generator seeded so that every run produces the same data. In the report, give this part one main figure and at most one page: what you changed, what happened, the mechanism behind it, whether it supported your hypothesis, and what you did not test.

Part 5: Report

Write a report of at most 5 pages containing your figures and tables, and answer these questions, using a mechanism ("why") and not just numbers:

1. At recall 0.95, which index is fastest on each dataset, and does the answer change between datasets?
2. What do the indexes cost in build time and memory? What does PQ save, and what does it give up?
3. What did your experiments in Parts 3 and 4 show, and what do they imply about when the headline numbers could mislead?
4. Suppose you have 150,000 vectors, or 10 million, a 100 ms budget per query and 16 GB of RAM. Using your exact-search timings and your results, would you use ANN, and if so which index and settings?
5. Name two ways your own benchmark could be misleading, and say whether you tested them.

Ground rules

You may use FAISS and hnswlib for the indexes, but write the recall function, timing code and plots yourself. Run everything on one machine and note its specs and library versions. Do not use the ann-benchmarks framework to run your experiments, and do not copy numbers from papers or other students.

Deliverables

Submit **one zip file named** `<rollnumber>_A3.zip` on Moodle. It must contain exactly the following, and nothing else. In particular, do not include the datasets or any index files.

1. `report.pdf`: your report from Part 5, at most 5 pages, with the figures and tables from Parts 1–4.
2. `code/`: a git repository (including its `.git` folder) with all your code (including your Part 4 data generator), a `README.md`, a `requirements.txt`, and your hypotheses for Parts 3 and 4 in `hypothesis.md`. The git history must show that each hypothesis was committed before the results of that experiment.
3. **One-command reproduction**: running `bash run_all.sh` inside `code/` must regenerate every number, figure and table in the report from the data files. Running `bash run_all.sh --quick` must do the same on a small subset of the data (for example the first 100,000 vectors) in under 15 minutes on a laptop. The README must explain how to run both.
4. `results/results.csv`: one row for every benchmark run behind a number in your report, and `results/figures/` with all your figures.
5. `ai_log/`: if you used any AI tool, all your prompts and the responses, plus a file `ai_statement.md` that marks every source file as human-written, AI-assisted or AI-generated. If you used no AI tools, `ai_log/` contains only `ai_statement.md` saying "No AI tools were used."

This is individual work. Late submissions follow the course policy.